2-学生体质信息管理系统【后端 REST API + 前端 Vue3/Vite/Axios】

后端的核心改动在 StuInfoController.java ，现在提供 GET/POST/PUT/DELETE /api/students 的 REST 接口；跨域配置在 WebMvcConfig.java ，统一异常返回在 GlobalExceptionHandler.java ，MVC JSON 支持和扫描配置在 spring-mvc.xml ，并且在 pom.xml 里补了 Jackson 依赖。

前端新建在 frontend/src/App.vue ，按模块拆成 api、components、样式文件；Axios 实例和后端地址在 http.js ，表单组件在 StudentForm.vue 。当前前端默认请求 http://localhost:8080/stu_pysical_ssm/api，如果你的 Tomcat 端口或上下文路径不同，改这个文件即可。

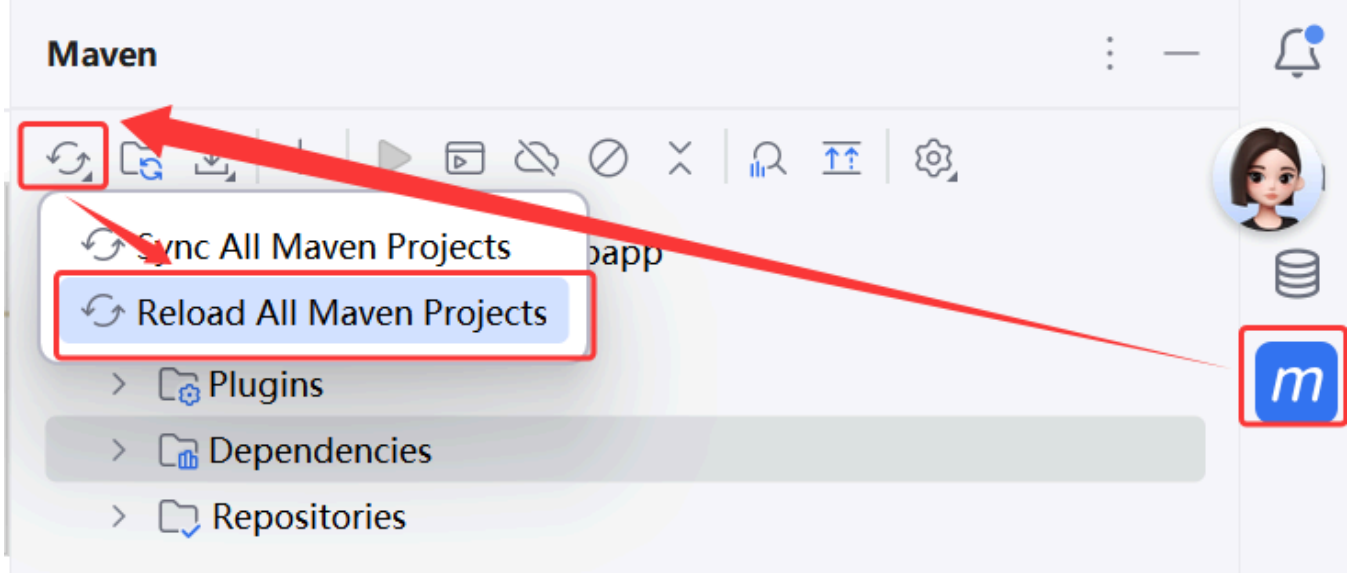
一. ##后端设计

1.1 添加依赖

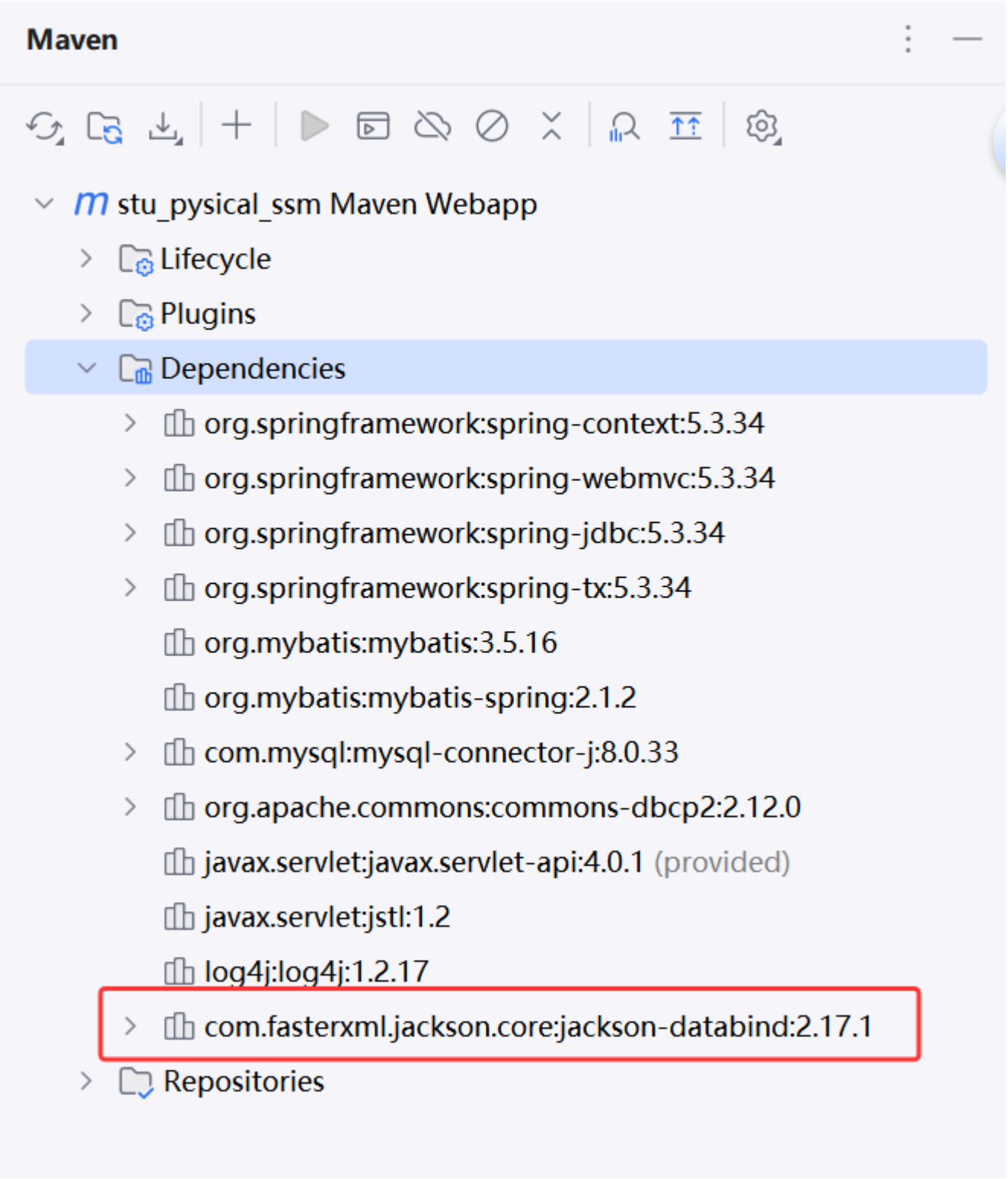
1、修改pom.xml ，增加Json 依赖。

```
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.1</version>
    </dependency>
```

2、重新加载依赖包



3、在Dependencies中查看是否多出Json的依赖包



1.2 REST API设计

在com.ssm包下创建dto包，在dto包下新建ApiResponse类

```
package com.ssm.dto;

public class ApiResponse<T> {
    private int code;
    private String message;
    private T data;
```

```java
    public ApiResponse() {
    }

    public ApiResponse(int code, String message, T data) {
        this.code = code;
        this.message = message;
        this.data = data;
    }

    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(200, message, data);
    }

    public static <T> ApiResponse<T> success(String message) {
        return new ApiResponse<>(200, message, null);
    }

    public static <T> ApiResponse<T> fail(String message) {
        return new ApiResponse<>(500, message, null);
    }

    public static <T> ApiResponse<T> fail(int code, String message) {
        return new ApiResponse<>(code, message, null);
    }

    public int getCode() {
        return code;
    }

    public void setCode(int code) {
        this.code = code;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public T getData() {
        return data;
    }

    public void setData(T data) {
        this.data = data;
    }
}
```

1.3 跨域配置

在com.ssm包下创建config包，在config包下新建 WebMvcConfig.java

```
package com.ssm.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

//跨域配置类
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")  // 匹配接口路径 /api/students
                .allowedOriginPatterns("*")     // 允许所有来源
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")     // 允许的请求方
式
                .allowedHeaders("*")    // 允许所有请求头
                .allowCredentials(false)
                .maxAge(3600);
    }
}
```

1.4 统一异常返回

在com.ssm包下创建exception包，在exception包下新建GlobalExceptionHandler类

```
package com.ssm.exception;

import com.ssm.dto.ApiResponse;
import org.springframework.dao.DuplicateKeyException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class)
    public ApiResponse<Void> handleIllegalArgument(IllegalArgumentException ex) {
        return ApiResponse.fail(400, ex.getMessage());
    }

    @ExceptionHandler(DuplicateKeyException.class)
    public ApiResponse<Void> handleDuplicate(DuplicateKeyException ex) {
        return ApiResponse.fail(409, "学号已存在，请勿重复新增");
    }

    @ExceptionHandler(Exception.class)
    public ApiResponse<Void> handleException(Exception ex) {
```

```
        return ApiResponse.fail(500, "服务器处理异常: " + ex.getMessage());
    }
}
```

1.5 修改controller和配置(spring-mvc.xml)

1、修改StuInfoController

```
package com.ssm.controller;

import com.ssm.dto.ApiResponse;
import com.ssm.po.StuInfo;
import com.ssm.service.StuInfoService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StuInfoController {

    @Autowired
    private StuInfoService stuInfoService;

    @GetMapping
    public ApiResponse<List<StuInfo>> list(@RequestParam(value = "keyword", required =
false) String keyword) {
        return ApiResponse.success("查询成功", stuInfoService.findByKeyword(keyword));
    }

    @GetMapping("/{stuNo}")
    public ApiResponse<StuInfo> detail(@PathVariable("stuNo") String stuNo) {
        StuInfo stuInfo = stuInfoService.findByStuNo(stuNo);
        if (stuInfo == null) {
            return ApiResponse.fail(404, "未找到对应学号的学生信息");
        }
        return ApiResponse.success("查询成功", stuInfo);
    }

    @PostMapping
    public ApiResponse<StuInfo> create(@RequestBody StuInfo stuInfo) {
        validateStudent(stuInfo, false);
        boolean ok = stuInfoService.save(stuInfo);
```

```java
        if (!ok) {
            return ApiResponse.fail("新增失败");
        }
        return ApiResponse.success("新增成功", stuInfo);
    }

    @PutMapping("/{stuNo}")
    public ApiResponse<StuInfo> update(@PathVariable("stuNo") String stuNo, @RequestBody
StuInfo stuInfo) {
        validateStudent(stuInfo, true);
        stuInfo.setStuNo(stuNo);
        boolean ok = stuInfoService.updateByStuNo(stuInfo);
        if (!ok) {
            return ApiResponse.fail(404, "修改失败，未找到对应学号");
        }
        return ApiResponse.success("修改成功", stuInfo);
    }

    @DeleteMapping("/{stuNo}")
    public ApiResponse<Void> delete(@PathVariable("stuNo") String stuNo) {
        boolean ok = stuInfoService.removeByStuNo(stuNo);
        if (!ok) {
            return ApiResponse.fail(404, "删除失败，未找到对应学号");
        }
        return ApiResponse.success("删除成功");
    }

    private void validateStudent(StuInfo stuInfo, boolean update) {
        if (stuInfo == null) {
            throw new IllegalArgumentException("请求体不能为空");
        }
        if (!update && isBlank(stuInfo.getStuNo())) {
            throw new IllegalArgumentException("学号不能为空");
        }
        if (isBlank(stuInfo.getStuName())) {
            throw new IllegalArgumentException("姓名不能为空");
        }
        if (isBlank(stuInfo.getGender())) {
            throw new IllegalArgumentException("性别不能为空");
        }
        if (stuInfo.getAge() == null || stuInfo.getAge() <= 0) {
            throw new IllegalArgumentException("年龄必须大于 0");
        }
        if (isBlank(stuInfo.getClassName())) {
            throw new IllegalArgumentException("班级不能为空");
        }
        if (stuInfo.getHeight() == null || stuInfo.getHeight() <= 0) {
            throw new IllegalArgumentException("身高必须大于 0");
        }
        if (stuInfo.getWeight() == null || stuInfo.getWeight() <= 0) {
            throw new IllegalArgumentException("体重必须大于 0");
        }
        if (stuInfo.getScore() == null || stuInfo.getScore() < 0) {
```

```
            throw new IllegalArgumentException("体测成绩不能小于 0");
        }
    }

    private boolean isBlank(String value) {
        return value == null || value.trim().isEmpty();
    }
}
```

2、修改spring-mvc.xml文件

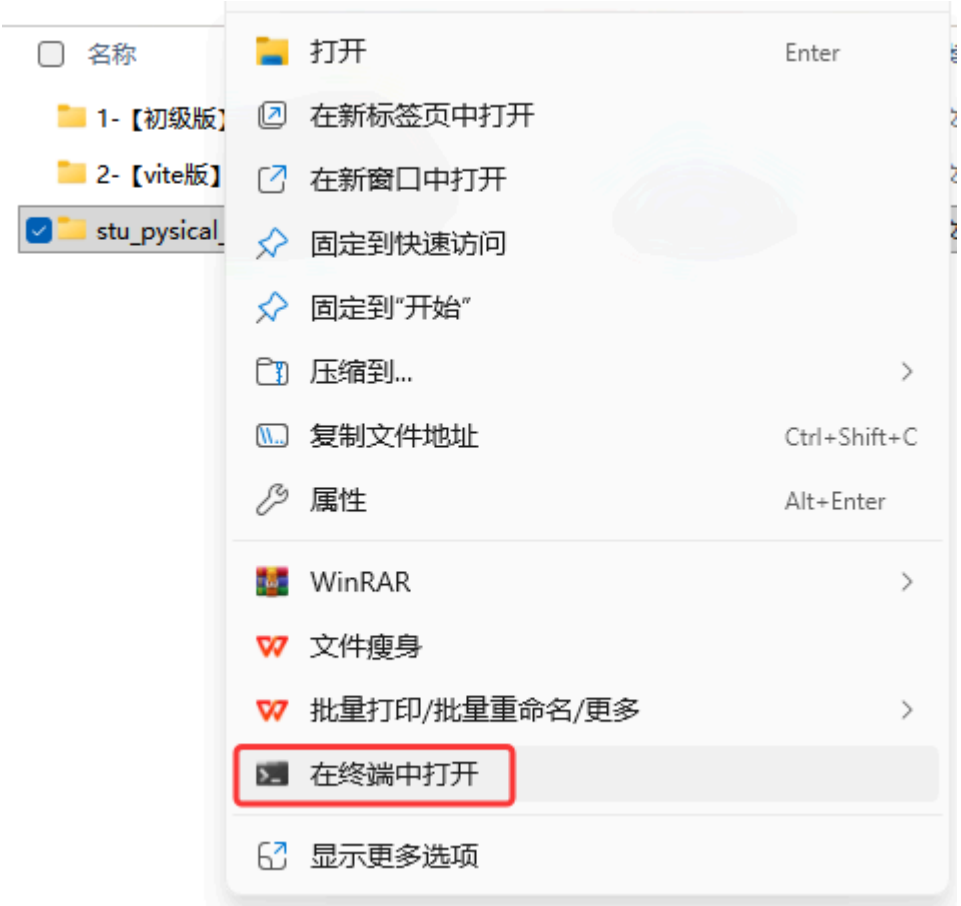
```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd
       http://www.springframework.org/schema/context
       http://www.springframework.org/schema/context/spring-context.xsd
       http://www.springframework.org/schema/mvc
       http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-
package="com.ssm.controller,com.ssm.config,com.ssm.exception"/>
    <mvc:annotation-driven />

    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/" />
        <property name="suffix" value=".jsp" />
    </bean>
</beans>
```

二. ##前端设计

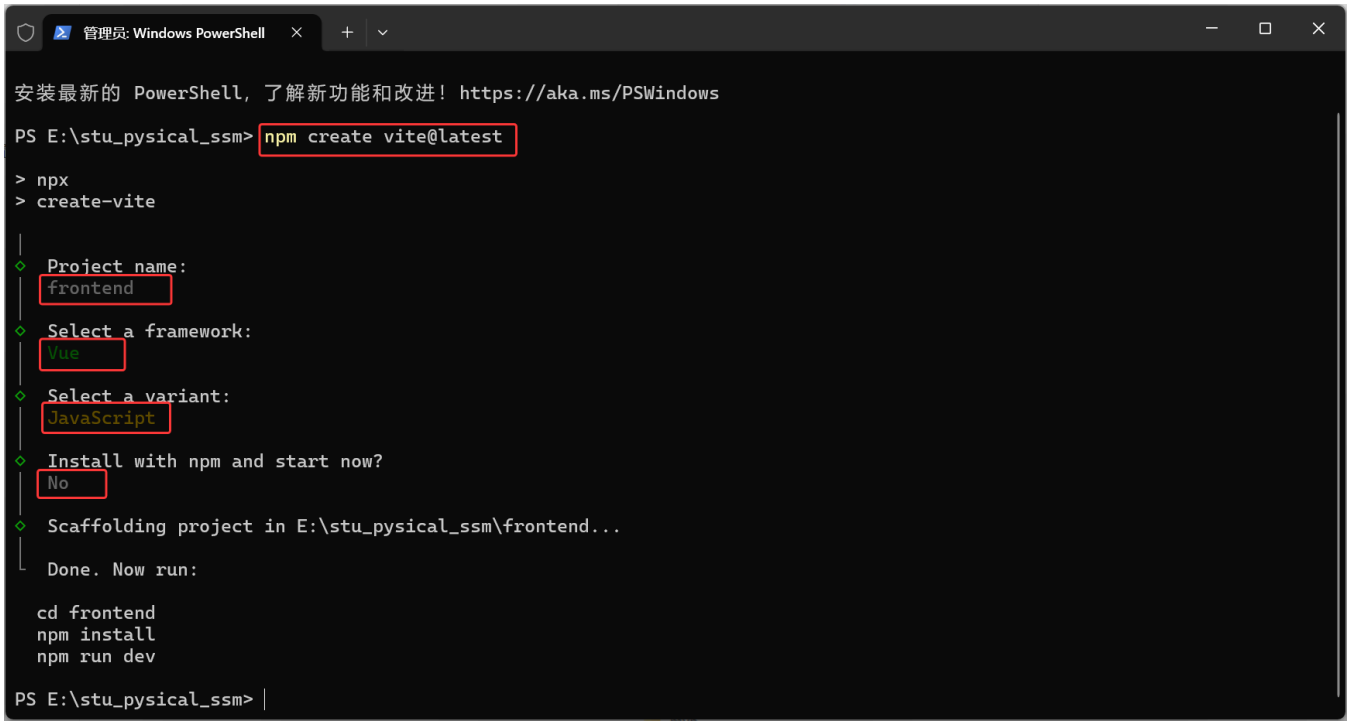
2.1 创建Vite项目

1、打开项目本地路径：stu_pysical_ssm

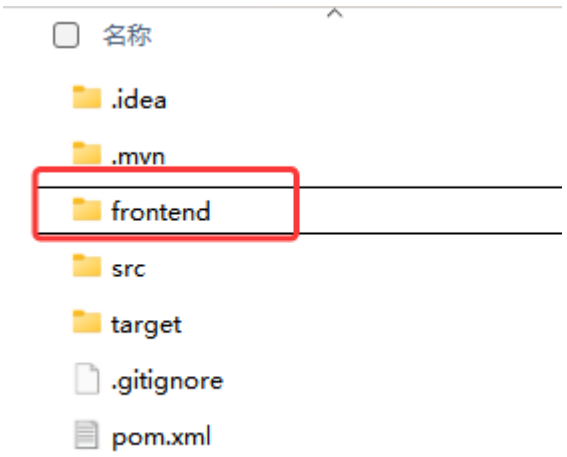
2、右击：stu_pysical_ssm，选择“在终端中打开”



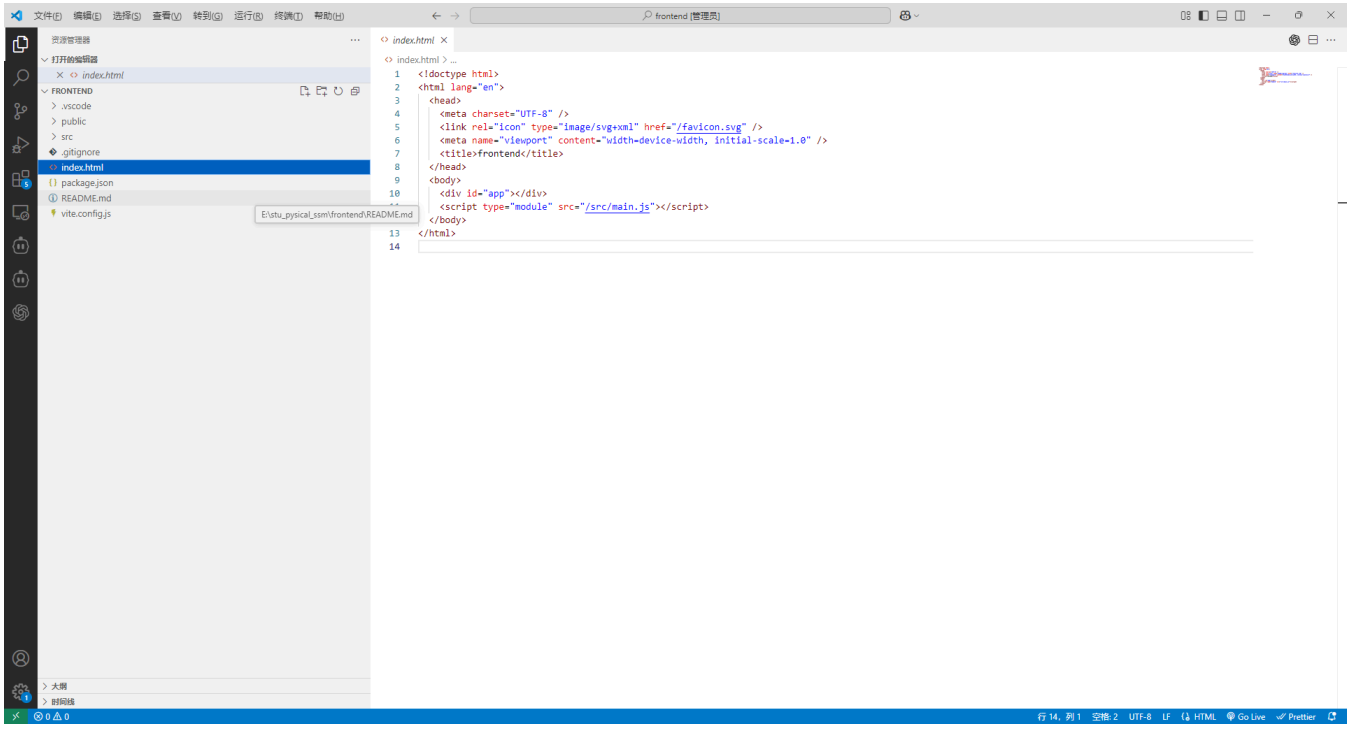
3、在终端中输入命令



4、在项目文件中查看，新增文件夹"frontend"



5、在vscode中打开frontend文件夹



2.2 前后端连接（代理/跨域）

`vite.config.js` 代理 + .env.development文件 + http.js文件，**共同完成前端请求后端接口**。

1、修改vite.config.js

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
   // 开发服务器配置（前端启动的服务）
  server: {
    host: '0.0.0.0',  // 允许局域网访问
    port: 5173,       // 前端运行端口：5173
    // 代理配置（解决跨域！）
    /**
     * 凡是前端访问 /api/xxx
```

```
     * 自动转发给后端 http://localhost:8080/stu_pysical_ssm/api/xxx
     */
    proxy: {
      '/api': {        //只要前端请求路径以 /api 开头，就走代理
        target: 'http://localhost:8080/stu_pysical_ssm',     //后端真实地址
        changeOrigin: true       //欺骗浏览器，假装是同源请求，解决跨域
      }
    }
  }
})
```

2、在根目录下，新建文件“.env.development”，保存配置。

```
# 存放后端地址
VITE_API_BASE_URL=/api
```

3、在src目录下，创建文件夹api，在api文件夹下新建http.js文件

```
// 统一发送请求
import axios from 'axios'

const baseURL = import.meta.env.VITE_API_BASE_URL || '/api'

const http = axios.create({
  baseURL,
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json'
  }
})

http.interceptors.response.use(
  (response) => {
    const result = response.data
    if (result.code !== 200) {
      return Promise.reject(new Error(result.message || '请求失败'))
    }
    return result
  },
  (error) => {
    const message =
      error.response?.data?.message ||
      error.message ||
      '网络异常，请检查后端服务是否启动'
    return Promise.reject(new Error(message))
  }
)

export default http
```

4、在api文件夹下，新建student.js文件

```
//调用 https.js 请求后端

import http from './http'

//  学生管理的所有后端接口统一封装文件
// 对应“StuInfoController”中，@RequestMapping("/api/students")
export function fetchStudents(keyword = '') {
  return http.get('/students', {
    params: { keyword }
  })
}

export function fetchStudent(stuNo) {
  return http.get(`/students/${stuNo}`)
}

export function createStudent(payload) {
  return http.post('/students', payload)
}

export function updateStudent(stuNo, payload) {
  return http.put(`/students/${stuNo}`, payload)
}

export function deleteStudent(stuNo) {
  return http.delete(`/students/${stuNo}`)
}
```

2.3 组件设计

1、在components下，新建SearchBar.vue

```
<script setup>
import { ref, watch } from 'vue'

const props = defineProps({
  loading: {
    type: Boolean,
    default: false
  }
})

const emit = defineEmits(['search', 'reset'])

const keyword = ref('')

watch(keyword, (value) => {
  if (!value) {
    emit('search', '')
  }
})
```

```
function onSearch() {
  emit('search', keyword.value.trim())
}

function onReset() {
  keyword.value = ''
  emit('reset')
}
</script>

<template>
  <section class="panel toolbar">
    <div class="toolbar__content">
      <div>
        <p class="eyebrow">检索操作</p>
        <h2>按学号或姓名查询学生</h2>
      </div>
      <div class="toolbar__form">
        <input
          v-model="keyword"
          id="student-keyword"
          name="keyword"
          class="input"
          type="text"
          placeholder="输入学号或姓名关键字"
          @keyup.enter="onSearch"
        />
        <button class="button button--primary" :disabled="props.loading" @click="onSearch">
          {{ props.loading ? '查询中...' : '查询' }}
        </button>
        <button class="button button--ghost" :disabled="props.loading" @click="onReset">重置
</button>
      </div>
    </div>
  </section>
</template>
```

2、在components下，新建StudentForm.vue

```
<script setup>
import { computed, reactive, watch } from 'vue'

const props = defineProps({
  modelValue: {
    type: Object,
    required: true
  },
  submitting: {
    type: Boolean,
    default: false
  },
  mode: {
    type: String,
```

```
    default: 'create'
  }
})

const emit = defineEmits(['update:modelValue', 'submit', 'cancel'])

const localForm = reactive(createDefaultForm())

watch(
  () => props.modelValue,
  (value) => {
    Object.assign(localForm, createDefaultForm(), value || {})
  },
  { immediate: true, deep: true }
)

watch(
  localForm,
  () => {
    emit('update:modelValue', { ...localForm })
  },
  { deep: true }
)

const title = computed(() => (props.mode === 'edit' ? '编辑学生信息' : '新增学生信息'))
const submitText = computed(() => {
  if (props.submitting) {
    return props.mode === 'edit' ? '保存中...' : '提交中...'
  }
  return props.mode === 'edit' ? '保存修改' : '立即新增'
})

function handleSubmit() {
  emit('submit')
}

function createDefaultForm() {
  return {
    stuNo: '',
    stuName: '',
    gender: '男',
    age: 18,
    className: '',
    height: 170,
    weight: 60,
    score: 80
  }
}
</script>

<template>
  <section class="panel form-panel">
    <div class="panel__header">
```

```
      <div>
        <p class="eyebrow">{{ props.mode === 'edit' ? '更新模式' : '录入模式' }}</p>
        <h2>{{ title }}</h2>
      </div>
      <button
        v-if="props.mode === 'edit'"
        class="button button--ghost"
        :disabled="props.submitting"
        @click="emit('cancel')"
      >
        取消编辑
      </button>
    </div>
    <div class="form-grid">
      <label class="field">
        <span>学号</span>
        <input v-model="localForm.stuNo" id="stuNo" name="stuNo" class="input"
:disabled="props.mode === 'edit'" type="text" />
      </label>
      <label class="field">
        <span>姓名</span>
        <input v-model="localForm.stuName" id="stuName" name="stuName" class="input"
type="text" />
      </label>
      <label class="field">
        <span>性别</span>
        <select v-model="localForm.gender" id="gender" name="gender" class="input">
          <option value="男">男</option>
          <option value="女">女</option>
        </select>
      </label>
      <label class="field">
        <span>年龄</span>
        <input v-model.number="localForm.age" id="age" name="age" class="input"
type="number" min="1" />
      </label>
      <label class="field">
        <span>班级</span>
        <input v-model="localForm.className" id="className" name="className" class="input"
type="text" />
      </label>
      <label class="field">
        <span>身高(cm)</span>
        <input v-model.number="localForm.height" id="height" name="height" class="input"
type="number" min="1" step="0.1" />
      </label>
      <label class="field">
        <span>体重(kg)</span>
        <input v-model.number="localForm.weight" id="weight" name="weight" class="input"
type="number" min="1" step="0.1" />
      </label>
      <label class="field">
        <span>体测成绩</span>
```

```
        <input v-model.number="localForm.score" id="score" name="score" class="input"
type="number" min="0" step="0.1" />
      </label>
    </div>
    <button class="button button--primary button--block" :disabled="props.submitting"
@click="handleSubmit">
      {{ submitText }}
    </button>
  </section>
</template>
```

3、在components下，新建StudentTable.vue

```
<script setup>
const props = defineProps({
  students: {
    type: Array,
    default: () => []
  },
  loading: {
    type: Boolean,
    default: false
  }
})

const emit = defineEmits(['edit', 'delete'])
</script>

<template>
  <section class="panel">
    <div class="panel__header">
      <div>
        <p class="eyebrow">数据视图</p>
        <h2>学生体测数据列表</h2>
      </div>
      <span class="badge">共 {{ props.students.length }} 条</span>
    </div>

    <div v-if="props.loading" class="empty-state">正在加载数据...</div>
    <div v-else-if="!props.students.length" class="empty-state">暂无数据，先新增一条学生记录吧。
</div>
    <div v-else class="table-wrap">
      <table class="table">
        <thead>
          <tr>
            <th>学号</th>
            <th>姓名</th>
            <th>性别</th>
            <th>年龄</th>
            <th>班级</th>
            <th>身高</th>
            <th>体重</th>
            <th>成绩</th>
```

```
            <th>操作</th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="student in props.students" :key="student.stuNo">
            <td>{{ student.stuNo }}</td>
            <td>{{ student.stuName }}</td>
            <td>{{ student.gender }}</td>
            <td>{{ student.age }}</td>
            <td>{{ student.className }}</td>
            <td>{{ student.height }}</td>
            <td>{{ student.weight }}</td>
            <td>{{ student.score }}</td>
            <td class="actions">
              <button class="button button--small button--ghost" @click="emit('edit',
student.stuNo)">编辑</button>
              <button class="button button--small button--danger" @click="emit('delete',
student.stuNo)">删除</button>
            </td>
          </tr>
        </tbody>
      </table>
    </div>
  </section>
</template>
```

2.4 App.vue

```
<script setup>
import { computed, onMounted, reactive, ref } from 'vue'
import SearchBar from './components/SearchBar.vue'
import StudentForm from './components/StudentForm.vue'
import StudentTable from './components/StudentTable.vue'
import { createStudent, deleteStudent, fetchStudent, fetchStudents, updateStudent } from
'./api/student'

const students = ref([])
const loading = ref(false)
const submitting = ref(false)
const keyword = ref('')
const mode = ref('create')
const message = reactive({
  type: 'success',
  text: ''
})
const form = ref(createDefaultForm())

const averageScore = computed(() => {
  if (!students.value.length) {
    return '0.0'
  }
  const total = students.value.reduce((sum, item) => sum + Number(item.score || 0), 0)
```

```
  return (total / students.value.length).toFixed(1)
})

onMounted(() => {
  loadStudents()
})

async function loadStudents(search = keyword.value) {
  loading.value = true
  keyword.value = search
  try {
    const result = await fetchStudents(search)
    students.value = result.data || []
  } catch (error) {
    showMessage('error', error.message)
  } finally {
    loading.value = false
  }
}

async function submitForm() {
  submitting.value = true
  try {
    const payload = { ...form.value }
    if (mode.value === 'edit') {
      await updateStudent(payload.stuNo, payload)
      showMessage('success', `学号 ${payload.stuNo} 的信息已更新`)
    } else {
      await createStudent(payload)
      showMessage('success', `学号 ${payload.stuNo} 已成功新增`)
    }
    resetForm()
    await loadStudents()
  } catch (error) {
    showMessage('error', error.message)
  } finally {
    submitting.value = false
  }
}

async function handleEdit(stuNo) {
  try {
    const result = await fetchStudent(stuNo)
    form.value = {
      ...createDefaultForm(),
      ...result.data
    }
    mode.value = 'edit'
    showMessage('success', `已载入学号 ${stuNo} 的信息，可以开始编辑`)
  } catch (error) {
    showMessage('error', error.message)
  }
}
```

```
async function handleDelete(stuNo) {
  const confirmed = window.confirm(`确定删除学号 ${stuNo} 的学生信息吗？`)
  if (!confirmed) {
    return
  }

  try {
    await deleteStudent(stuNo)
    showMessage('success', `学号 ${stuNo} 已删除`)
    if (mode.value === 'edit' && form.value.stuNo === stuNo) {
      resetForm()
    }
    await loadStudents(keyword.value)
  } catch (error) {
    showMessage('error', error.message)
  }
}

function handleSearch(value) {
  loadStudents(value)
}

function resetForm() {
  form.value = createDefaultForm()
  mode.value = 'create'
}

function showMessage(type, text) {
  message.type = type
  message.text = text
}

function createDefaultForm() {
  return {
    stuNo: '',
    stuName: '',
    gender: '男',
    age: 18,
    className: '',
    height: 170,
    weight: 60,
    score: 80
  }
}
</script>

<template>
  <div class="page-shell">
    <header class="hero">
      <div class="hero__text">
        <p class="eyebrow">Vue 3 + Vite + Axios + Spring MVC REST API</p>
        <h1>学生体测信息管理系统</h1>
```

```
        <p class="hero__desc">
          前端负责交互与视图渲染，后端专注提供 REST 接口。现在这套项目已经改造成适合课程演示的前后端分离结构。
        </p>
      </div>
      <div class="hero__stats">
        <article class="stat-card">
          <span>学生总数</span>
          <strong>{{ students.length }}</strong>
        </article>
        <article class="stat-card">
          <span>平均体测成绩</span>
          <strong>{{ averageScore }}</strong>
        </article>
      </div>
    </header>

    <p v-if="message.text" :class="['message', `message--${message.type}`]">
      {{ message.text }}
    </p>

    <SearchBar :loading="loading" @search="handleSearch" @reset="loadStudents('')" />

    <main class="layout-grid">
      <StudentForm
        v-model="form"
        :mode="mode"
        :submitting="submitting"
        @submit="submitForm"
        @cancel="resetForm"
      />
      <StudentTable :students="students" :loading="loading" @edit="handleEdit" @delete="handleDelete" />
    </main>
  </div>
</template>
```

2.5 修改style.css文件

```
:root {
  color-scheme: light;
  font-family: "Microsoft YaHei", "PingFang SC", sans-serif;
  color: #16324f;
  background:
    radial-gradient(circle at top left, rgba(251, 191, 36, 0.28), transparent 28%),
    radial-gradient(circle at top right, rgba(14, 165, 233, 0.18), transparent 24%),
    linear-gradient(135deg, #f4efe6 0%, #f3f8fc 48%, #eef6f0 100%);
}

* {
  box-sizing: border-box;
}
```

```
body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
}

button,
input,
select {
  font: inherit;
}

#app {
  min-height: 100vh;
}

.page-shell {
  width: min(1200px, calc(100% - 32px));
  margin: 0 auto;
  padding: 32px 0 48px;
}

.hero {
  display: grid;
  grid-template-columns: 1.5fr 0.9fr;
  gap: 20px;
  align-items: stretch;
  margin-bottom: 24px;
}

.hero__text,
.hero__stats,
.panel {
  border: 1px solid rgba(22, 50, 79, 0.08);
  border-radius: 24px;
  background: rgba(255, 255, 255, 0.8);
  backdrop-filter: blur(16px);
  box-shadow: 0 20px 45px rgba(60, 72, 88, 0.08);
}

.hero__text {
  padding: 32px;
}

.hero__text h1 {
  margin: 8px 0 14px;
  font-size: clamp(2rem, 4vw, 3.5rem);
  line-height: 1.1;
}

.hero__desc {
  margin: 0;
```

```
  max-width: 700px;
  color: #4b5d70;
  font-size: 1rem;
}

.hero__stats {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
  padding: 24px;
}

.stat-card {
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 10px;
  border-radius: 20px;
  padding: 22px;
  background: linear-gradient(135deg, #16324f, #235784);
  color: #f8fafc;
}

.stat-card strong {
  font-size: 2rem;
}

.eyebrow {
  margin: 0;
  color: #b45309;
  font-size: 0.85rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}

.panel {
  padding: 24px;
}

.toolbar {
  margin-bottom: 24px;
}

.toolbar__content,
.panel__header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}

.toolbar__content h2,
```

```
.panel__header h2 {
  margin: 8px 0 0;
}

.toolbar__form {
  display: flex;
  gap: 12px;
  align-items: center;
  flex-wrap: wrap;
}

.layout-grid {
  display: grid;
  grid-template-columns: minmax(320px, 380px) minmax(0, 1fr);
  gap: 24px;
  align-items: start;
}

.form-panel {
  position: sticky;
  top: 24px;
}

.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
  margin: 20px 0;
}

.field {
  display: flex;
  flex-direction: column;
  gap: 8px;
  color: #334155;
  font-size: 0.95rem;
}

.input {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid rgba(71, 85, 105, 0.18);
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.9);
  outline: none;
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

.input:focus {
  border-color: #0f766e;
  box-shadow: 0 0 0 4px rgba(15, 118, 110, 0.12);
}
```

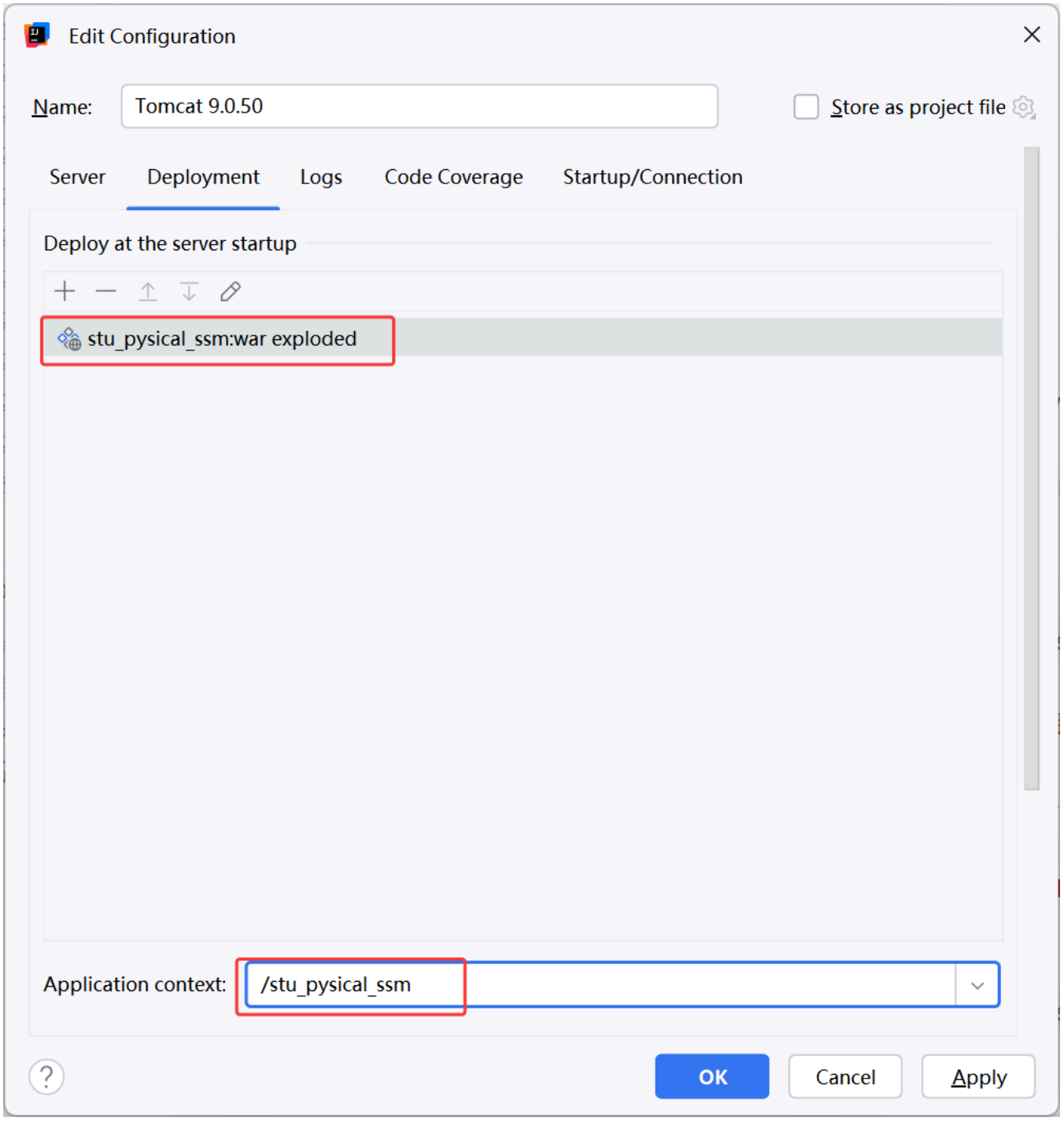
```
.button {
  border: none;
  border-radius: 14px;
  padding: 12px 18px;
  cursor: pointer;
  transition: transform 0.18s ease, opacity 0.18s ease;
}

.button:hover {
  transform: translateY(-1px);
}

.button:disabled {
  cursor: not-allowed;
  transform: none;
  opacity: 0.65;
}

.button--primary {
  background: linear-gradient(135deg, #0f766e, #0891b2);
  color: #fff;
}

.button--ghost {
  background: #eef2f7;
  color: #1f2937;
}

.button--danger {
  background: #ef4444;
  color: #fff;
}

.button--small {
  padding: 8px 12px;
  border-radius: 10px;
}

.button--block {
  width: 100%;
}

.badge {
  padding: 8px 14px;
  border-radius: 999px;
  background: #fff7ed;
  color: #c2410c;
  font-weight: 700;
}

.table-wrap {
  overflow-x: auto;
}
```

```
.table {
  width: 100%;
  border-collapse: collapse;
}

.table th,
.table td {
  padding: 14px 12px;
  text-align: left;
  border-bottom: 1px solid rgba(148, 163, 184, 0.18);
  white-space: nowrap;
}

.table th {
  color: #475569;
  font-size: 0.9rem;
}

.actions {
  display: flex;
  gap: 8px;
}

.empty-state {
  padding: 48px 0;
  text-align: center;
  color: #64748b;
}

.message {
  margin: 0 0 24px;
  padding: 14px 18px;
  border-radius: 16px;
  font-weight: 600;
}

.message--success {
  background: rgba(34, 197, 94, 0.14);
  color: #166534;
}

.message--error {
  background: rgba(239, 68, 68, 0.12);
  color: #b91c1c;
}

@media (max-width: 960px) {
  .hero,
  .layout-grid {
    grid-template-columns: 1fr;
  }
```

```
  .form-panel {
    position: static;
  }
}

@media (max-width: 640px) {
  .page-shell {
    width: min(100% - 20px, 1200px);
    padding-top: 20px;
  }

  .hero__text,
  .hero__stats,
  .panel {
    padding: 18px;
    border-radius: 20px;
  }

  .toolbar__content,
  .panel__header {
    flex-direction: column;
    align-items: flex-start;
  }

  .form-grid {
    grid-template-columns: 1fr;
  }
}
```

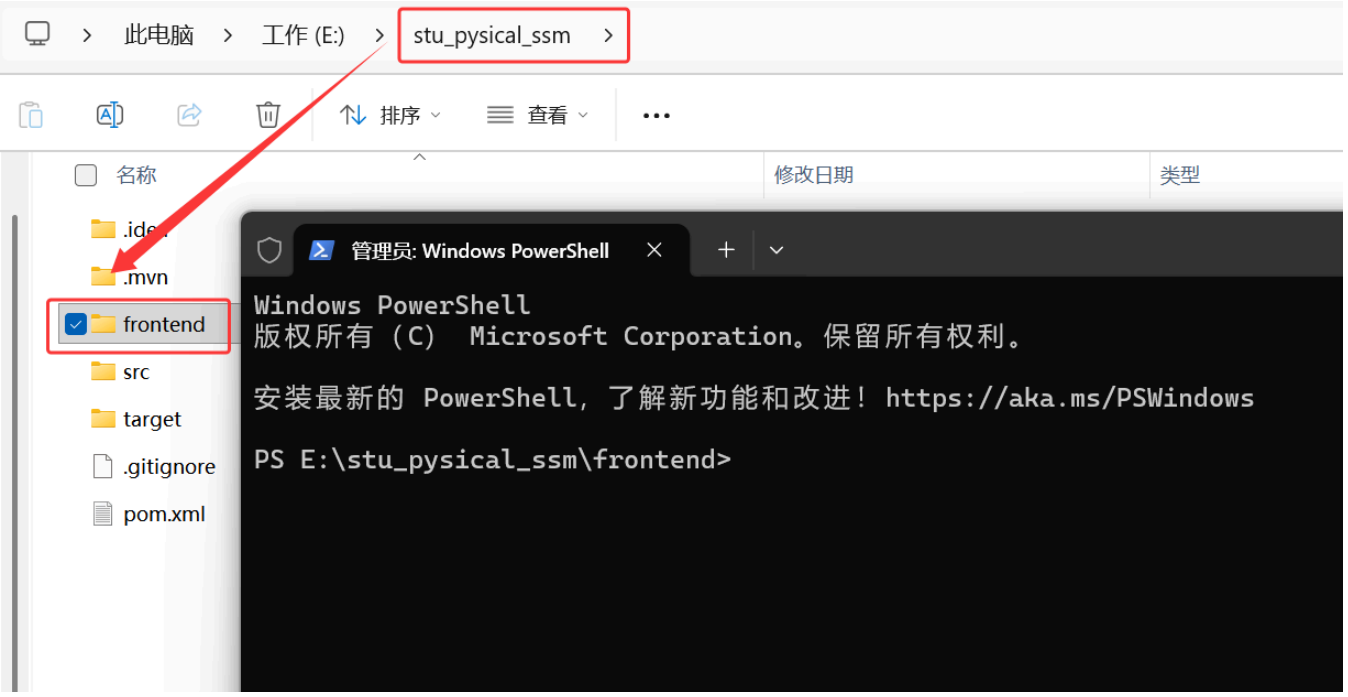
三. ##部署运行

3.1 后端部署，启动tomcat服务器。



3.2 前端运行

打开“stu_pysical_ssm”文件夹，找到“frontend”，右击“frontend”文件夹，选择“在终端中打开”



在终端中输入：

```
npm install
npm install axios
npm run dev
```

打开链接Local: http://localhost:5173/，查看运行页面

四. ##自行功能测试